



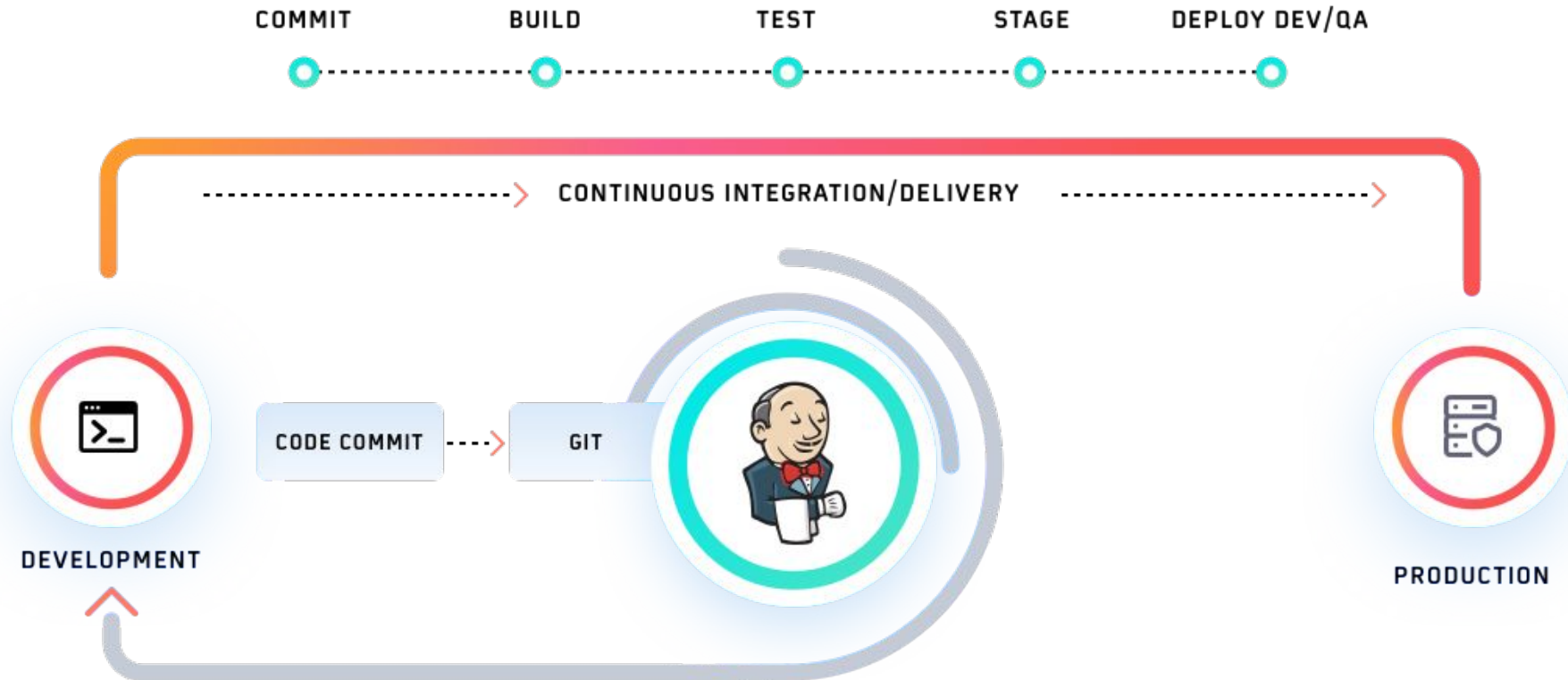
DevOps



Jenkins Pipeline

What is Jenkins Pipeline?

- **Jenkins pipeline** in CI/CD automates many tasks and makes the CI/CD pipeline tasks reliable, efficient, repeatable, and high in quality. It defines a complete list of events that happen in the code lifecycle. Starting from the build, to testing and deployment.



Scripted vs. Declarative Pipelines

- Jenkins supports two types of pipeline definition.
 - **Scripted pipelines** which are written in Groovy language.
 - **Declarative pipelines** which are written in **Jenkins DSL** language.
- While both of these pipelines are Groovy-based, the scripted pipeline uses more strict Groovy-based syntaxes. This is because it was the first groovy foundation pipeline that was created for use. As this Groovy script was not usually suitable to all users, it introduced the declarative pipeline to provide a simpler and more flexible Groovy syntax. The declarative pipeline is defined within a 'pipeline' block, while the scripted pipeline is defined within a 'node' block.
- We intend to talk about the Jenkins DSL language which is more readable and easy to learn. Furthermore, within this DSL language, we can write Groovy scripts too.

Scripted vs. Declarative Pipelines

```
node("worker1") {  
  stage('Build') {  
    echo 'building the application...'  
  }  
  stage('Test') {  
    echo 'testing the application...'  
  }  
  stage('Deploy') {  
    echo 'deploying the application...'  
  }  
}
```

Scripted Pipeline

```
pipeline {  
  agent any  
  stages {  
    stage('Build') {  
      steps {  
        echo 'building the application...'  
      }  
    }  
    stage('Test') {  
      steps {  
        echo 'building the application...'  
      }  
    }  
    stage('Deploy') {  
      steps {  
        echo 'building the application...'  
      }  
    }  
  }  
}
```

Declarative Pipeline

Pipeline with ReactJS

```
pipeline{
  agent{
    label 'master'
  }
  tools {
    nodejs '16.18.1'
  }
  stages{
    stage("Checkout Git Resource"){
      steps{
        echo "Checkout Git Resource on node ${NODE_NAME} and $NODE_LABELS";
        git 'https://github.com/leangsengk90/seng-react-test.git';
      }
    }
  }
}
```

Pipeline with ReactJS

```
stage("Build in Jenkins"){
    steps{
        echo "Build in Jenkins";
        sh 'npm install --force';
        sh 'npm run build';
    }
}

stage("Docker Compose Stage"){
    steps{
        echo "Docker Compose Stage";
        // sh 'docker compose down';
        sh 'docker compose up -d --build';
    }
}
}
```

Pipeline with Spring(Jenkinsfile)

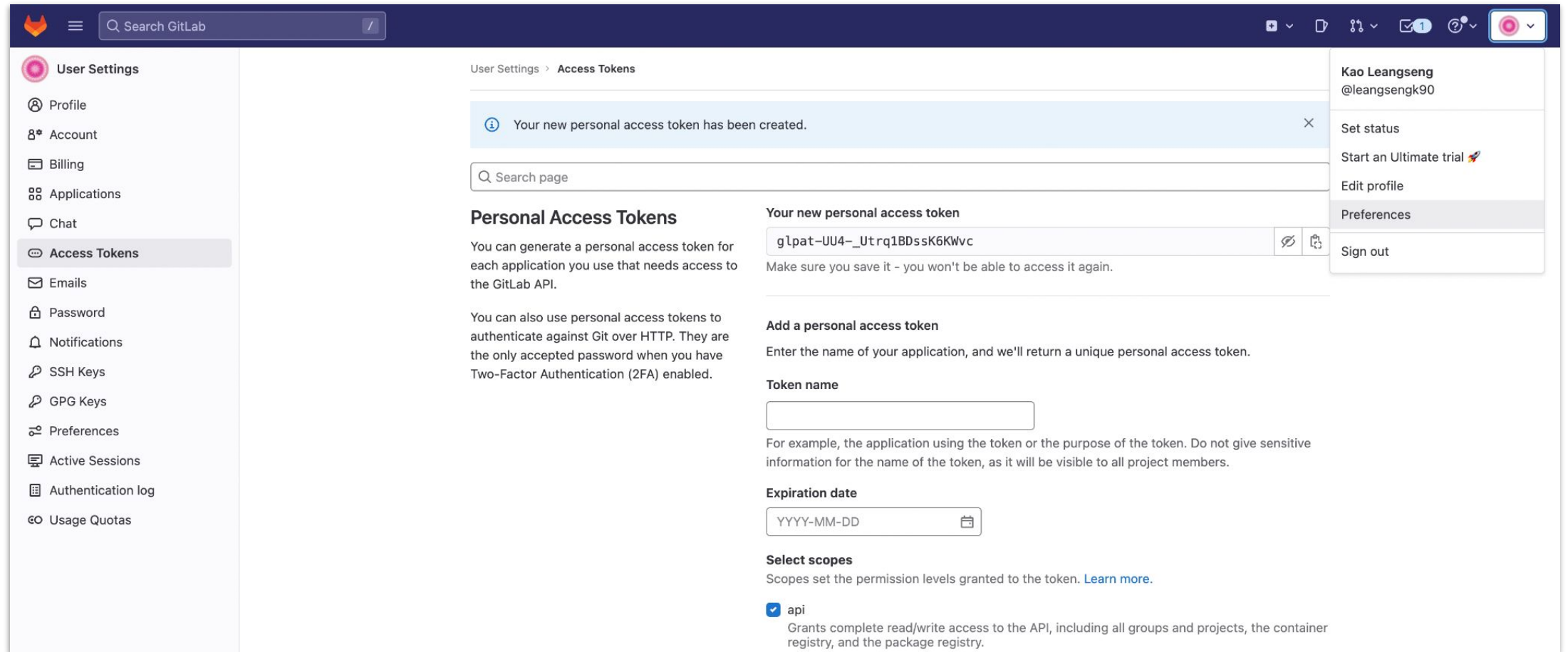
```
pipeline{
  agent{
    label 'worker1'
  }
  tools {
    maven '3.6.3'
  }
  stages{
    stage("Pull Git Repo"){
      steps{
        echo "Pull Git Repo on node: $NODE_NAME"
        // git credentialsId: 'a39667f0-0833-4713-866b-f5a0d8e5ca05', url:
        'https://github.com/leangsengk90/spring-sample.git'
      }
    }
  }
}
```

Pipeline with Spring(Jenkinsfile)

```
stage("Build with Maven"){
    steps{
        echo "Build with Maven"
        sh 'mvn clean package'
        sh 'docker build -t spring-demo-img .'
    }
}
stage("Deploy Container"){
    steps{
        script{
            try{
                sh 'docker rm -f spring-demo-cont'
            }catch(err){
                echo "ERR:"+err;
            }
        }
        sh 'docker run -d -p 8282:8080 --name spring-demo-cont spring-demo-img'
    }
}
```


Gitlab Webhook

- Get token from Gitlab



User Settings

- Profile
- Account
- Billing
- Applications
- Chat
- Access Tokens**
- Emails
- Password
- Notifications
- SSH Keys
- GPG Keys
- Preferences
- Active Sessions
- Authentication log
- Usage Quotas

User Settings > Access Tokens

Search GitLab

Search page

Personal Access Tokens

You can generate a personal access token for each application you use that needs access to the GitLab API.

You can also use personal access tokens to authenticate against Git over HTTP. They are the only accepted password when you have Two-Factor Authentication (2FA) enabled.

Your new personal access token

glpat-UU4-_Utrq1BDssK6KWvc

Make sure you save it - you won't be able to access it again.

Add a personal access token

Enter the name of your application, and we'll return a unique personal access token.

Token name

For example, the application using the token or the purpose of the token. Do not give sensitive information for the name of the token, as it will be visible to all project members.

Expiration date

YYYY-MM-DD

Select scopes

Scopes set the permission levels granted to the token. [Learn more.](#)

- ☒ **api**
Grants complete read/write access to the API, including all groups and projects, the container registry, and the package registry.

Kao Leangseng
@leangsengk90

- Set status
- Start an Ultimate trial
- Edit profile
- Preferences**
- Sign out

Gitlab Webhook

- Install Gitlab Plugin and Gitlab API Plugin

Gitlab API Plugin 5.0.1-78.v47a_45b_9f78b_7

This plugin provides **GitLab API** for other plugins.

[Report an issue with this plugin](#)

GitLab Plugin 1.6.0

This plugin allows **GitLab** to trigger Jenkins builds and display their results in the GitLab UI.

[Report an issue with this plugin](#)

Gitlab Webhook

- Manage Jenkins => Configure System

Gitlab

☒ Enable authentication for '/project' end-point

GitLab connections

Connection name ✕

A name for the connection

Gitlab host URL

The complete URL to the Gitlab server (e.g. http://gitlab.mydomain.com)

Credentials

API Token for accessing Gitlab

+ Add

Advanced...

Test Connection

Add

Gitlab Webhook

- Manage Jenkins => Manage Credentials

Jenkins Credentials Provider: Jenkins

Add Credentials

Domain

Global credentials (unrestricted) ▾

Kind

GitLab API token ▾

Scope ?

Global (Jenkins, nodes, items, all child items, etc) ▾

API token

.....

ID ?

Description ?

Pipeline with Docker Image

```
pipeline{
  agent{
    docker {
      image 'node:16.18.1'
      args '-u root'
    }
  }
  stages{
    stage("Get Git Repository"){
      steps{
        git 'https://github.com/leangsengk90/seng-react-test.git'
      }
    }
    stage("Build and Deploy"){
      steps{
        sh 'npm install --force'
        sh 'npm run build'
        sh 'docker compose up -d --build'
      }
    }
  }
}
```

Pipeline with Docker Image

```
node("worker-1") {
    //checkout scm
    stage("Git Resource"){
        git credentialsId: 'Gitlab-Authentication', url:
        'https://gitlab.com/leangsengk90/seng_registration_final.git'
    }
    stage("Build with Spring"){
        docker.image('maven:3.6.3').inside{
            sh 'mvn clean package'
        }
        docker.build("new-spring-register")
        //Testing
        // docker.image("new-spring-register").withRun('-p 8989:8080'){
        //     //sh 'docker ps -a'
        //     //This container will be remove after testing.
        // }
    }
    stage("Deploy Container"){
        sh 'docker run -d -p 8989:8080 --name new-spring-register-cont new-spring-register'
    }
}
```



Pipeline with Email Notification

- Go to: Your Gmail Profile Account => Manage Your Gmail Account

The image shows a Google Account 'Security' settings page on the left and a 'App passwords' generation dialog on the right.

Left Panel: Google Account Security Settings

- Home
- Personal info
- Data & privacy
- Security**
- People & sharing
- Payments & subscriptions
- About

Signing in to Google

- Password: Last changed Aug 20, 2017
- 2-Step Verification: ☒ On
- App passwords**: 1 password

Right Panel: App passwords

App passwords let you sign in to your Google Account from apps on devices that don't support 2-Step Verification. You'll only need to enter it once so you don't need to remember it. [Learn more](#)

Your app passwords

Name	Created	Last used	
Mail on my Mac	Jun 1	Oct 7	

Select the app and device you want to generate the app password for.

Mail

Select device

iPhone

iPad

BlackBerry

Mac

Windows Phone

Windows Computer

Other (Custom name)

GENERATE

Pipeline with Email Notification

- Manage Jenkins => Configure System

E-mail Notification

SMTP server

smtp.gmail.com

Default user e-mail suffix ?

☒ Use SMTP Authentication ?

User Name

leangsengk90@gmail.com

Password

.....

☒ Use SSL ?

☐ Use TLS

Generated app password

Your app password for Mac

elxm frei kpkt lmfx

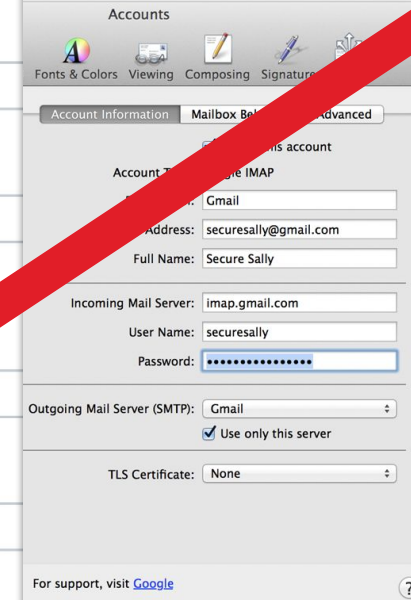
How to use it

1. Open the "Mail" app on your Mac computer.
2. From the Menu bar, select "Mail" → "Preferences..." → "Accounts".
3. Select your Google Account from the list.
4. Replace your password with the 16-character password shown above.

Just like your normal password, this app password grants complete access to your Google Account. You won't need to remember it, so don't write it down or share it with anyone.

[Learn more](#)

DONE



Pipeline with Email Notification

SMTP Port ?

465

Reply-To Address

leangsengk90@mail.com

Charset

UTF-8

☒ Test configuration by sending test e-mail

Test e-mail recipient

mommei830@gmail.com

Email was successfully sent

Test configuration

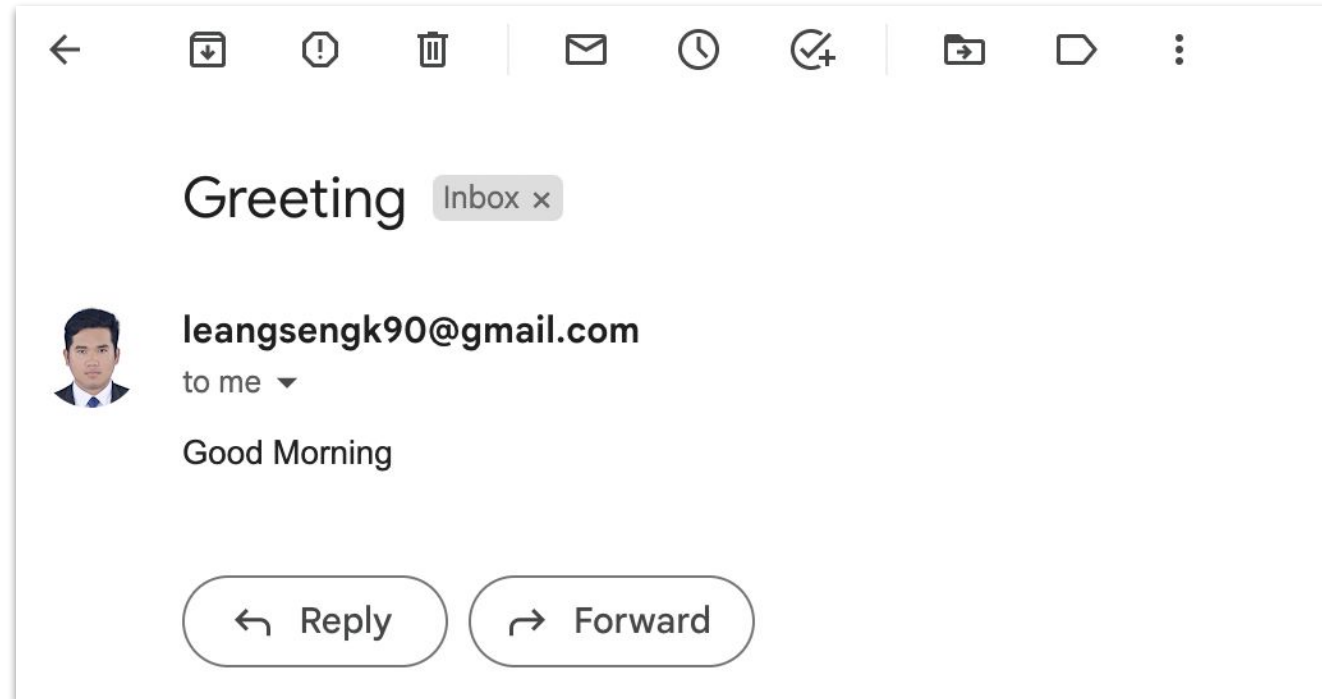
Save

Apply

Pipeline with Email Notification

- Generate pipeline syntax from Sample Step “mail: Mail”

```
stage("Deploy Container"){  
  sh 'docker run -d -p 8989:8080 --name new-spring-register-cont new-spring-register'  
  mail bcc: '', body: 'Good Morning', cc: '', from: 'leangsengk90@gmail.com', replyTo: '', subject: 'Greeting', to: 'mommei830@gmail.com'  
}
```



Pipeline with Environment Variable

- Built-in(Global) variable: <http://serve-ip:8080/env-vars.html/>
- Local variable, Global Variable(Configuration System)

```
pipeline {
  agent any
  environment{
    TODAY = "Monday" # Local variable
  }
  stages {
    stage('Hello') {
      steps {
        echo "TODAY is ${TODAY}"
        echo "Build Number: ${env.BUILD_NUMBER}" # Built-in variable
      }
    }
  }
}
```

Pipeline with Post Actions

```
post {  
    always {  
        echo "This block always runs."  
    }  
    changed {  
        echo "This block runs when the current status is different than the previous one."  
    }  
    fixed {  
        echo "This block runs when the current status is success and the previous one was  
failed or unstable."  
    }  
    regression {  
        echo "This block runs when the current status is anything except success but the  
previous one was successful."  
    }  
}
```

Pipeline with Post Actions

```
unstable {  
    echo "This block runs if the current status is marked unstable."  
}  
aborted {  
    echo "This block runs when the build process is aborted."  
}  
failure {  
    echo "This block runs when the build is failed."  
}  
success {  
    echo "This block runs when the build is succeeded."  
}  
unsuccessful {  
    echo "This block runs when the current status is anything except success."  
}  
cleanup {  
    echo "This block always runs after other conditions are evaluated."  
}  
}
```



Pipeline with Generic Webhook Trigger

```
pipeline {
  agent any
  triggers {
    GenericTrigger(
      genericVariables: [
        [key: 'ref', value: '$.ref']
      ],
      causeString: "Test GenericTrigger!",
      token: "123"
    )
    //cron "* * * * *"
  }
  stages {
    stage("Test") {
      steps {
        echo "Hello World, GenericTrigger!"
        //curl -i http://SENG:123@34.143.157.108:8080/generic-webhook-trigger/invoke\?token=123 2>&1
      }
    }
  }
}
```

Pipeline with Interactive and Parameter

```
stage("Git"){  
    input {  
        message "Ready to deploy?"  
        ok "Yes"  
        parameters {  
            booleanParam(name: "TEST_BOOLEAN", defaultValue: true, description: "Sample boolean parameter")  
            string(name: "TEST_STRING", defaultValue: "dara", trim: true, description: "Sample string parameter")  
            text(name: "TEST_TEXT", defaultValue: "Jenkins Pipeline Tutorial", description: "Sample multi-line  
                text parameter")  
            password(name: "TEST_PASSWORD", defaultValue: "SECRET", description: "Sample password parameter")  
            choice(name: "TEST_CHOICE", choices: ["production", "staging", "development"], description: "Sample  
                multi-choice parameter")  
        }  
    }  
}
```

Pipeline with Interactive and Parameter

```
steps{
  echo "TEST_CHOICE is ${TEST_CHOICE}"
  script{
    def deploy = "${TEST_CHOICE}"
    if(deploy == "production"){
      echo "Clone from master"
      git credentialsId: '416bfe8f-26d4-43fb-92c6-5b33891770d3', url:
        'https://github.com/leangsengk90/spring-sample.git'
    }else if(deploy == "development"){
      echo "Clone from testing"
      git branch: 'testing', credentialsId: '416bfe8f-26d4-43fb-92c6-5b33891770d3', url:
        'https://github.com/leangsengk90/spring-sample.git'
    }
  }
}
```


Options – Pipeline behavior

```
pipeline {
  agent any
  options{
    timeout(time: 5, unit: 'SECONDS')
    buildDiscarder logRotator(numToKeepStr: '3', daysToKeepStr: '7')
  }
  stages {
    stage('Stage One') {
      options{
        retry(3)
      }
      steps {
        script{
          sh 'sleep 3'
          throw new Exception("Terminate Pipeline...")
        }
        echo 'Hello One'
      }
      post{
        always{
          script{
            if(currentBuild.result == 'SUCCESS'){
              echo 'Hello SUCCESS'
            }else if(currentBuild.result == 'FAILURE'){
              echo 'Hello FAILURE'
            }else{
              echo 'somethings else'
            }
          }
        }
      }
    }
  }
}
```

}}}}}}



Parallel Stage

```
pipeline {
  agent any
  stages {
    stage("Wait 5 seconds") {
      steps {
        cleanWs()
        sh '''
            mkdir -p production
            sleep 5
            ...
        '''
      }
    }
    stage("Start Operation") {
      parallel {
        stage('Project Resource') {
          steps {
            dir("${WORKSPACE}/production") {
              sh 'touch resource.txt'
            }
          }
        }
      }
    }
  }
}
```



Parallel Stage

```
stage('Build') {
  steps {
    script{
      for (int i=1; i<=3; i++){
        sh "touch file${i}"
      }
    }
  }
}
stage('Deploy') {
  steps {
    sh """
      touch deploy.txt
      echo 'JOB: ${JOB_NAME}, BUILD: ${BUILD_NUMBER} deploy success!'
    """
  }
}
}
```



Blue Ocean Plugin

Jenkins

Dashboard > frontend > parallel-project >

Pipeline parallel-project

Full project name: frontend/parallel-project

Stage View

Average stage times:
(Average full run time: ~7s)

	Wait 5 seconds	Start Operation	Project Resource	Build	Deploy
#18 Mar 04 01:45 No Changes	5s	89ms	594ms	1s	421ms
#17 Mar 04 01:44 No Changes	5s	91ms	617ms	1s	434ms
#16 Mar 04 01:43 No Changes	5s	86ms	591ms	1s	409ms

Build History trend ▾

Filter builds... /

✓ #18
Mar 3, 2023, 6:45 PM

Blue Ocean Plugin

✓ frontend / parallel-project < 18

PipelineChangesTestsArtifacts🔄⚙️🔗Logout✕

Branch: —

🕒 7s

No changes

Commit: —

🕒 9 minutes ago

Started by user jenkins

```
graph LR; Start((Start)) --> Wait((Wait 5 seconds)); Wait --> SO((Start Operation)); SO --> End((End)); SO --> Build((Build)); SO --> Deploy((Deploy)); SO --> PR((Project Resource));
```

Start Operation / Deploy - <1s

[Restart Start Operation](#) 🗑️ ⬇️

✓

✓ touch deploy.txt echo "JOB: frontend/parallel-project, BUILD: 18 deploy successfully!" — Shell Script <1s

```
1 + touch deploy.txt
2 + echo JOB: frontend/parallel-project, BUILD: 18 deploy successfully!
3 JOB: frontend/parallel-project, BUILD: 18 deploy successfully!
```

Shared Library

leangsengk90 / my-shared-library

Public

Pin

Unwatch 1

<> Code

Issues

Pull requests

Actions

Projects

Wiki

Security

Insights

main

1 branch

0 tags

Go to file

Add file

<> Code

leangsengk90 Update deployContainer.groovy

bbbb96f 5 hours ago

26 commits

resources/scripts/frontend

Update deploy.sh

7 hours ago

src/docker/release

Update Deployer.groovy

5 hours ago

vars

Update deployContainer.groovy

5 hours ago

README.md

Initial commit

8 hours ago

Shared Library

 main ▾ my-shared-library / vars /

 leangsengk90 Update deployContainer.groovy

..

 calculate.groovy

 deployContainer.groovy

 getResource.groovy

 helloWorld.groovy

 main ▾ my-shared-library / src / docker / release /

 leangsengk90 Update Deployer.groovy

..

 Deployer.groovy Update Deplo

[Give feedback](#)

 main ▾ my-shared-library / resources / scripts / frontend /

 leangsengk90 Update deploy.sh

..

 deploy.sh Update deploy.sh

Shared Library

```
#helloWorld.groovy
#!/usr/bin/env groovy
def call(String name){
    sh "echo 'Hello World! Bro ${name}'"
}
```

```
#calculate.groovy
#!/usr/bin/env groovy
def sum(a, b){
    echo "SUM: $a + $b = ${a+b}"
}
def minus(a, b){
    echo "MINUS: $a - $b = ${a-b}"
}
```


Shared Library

```
#deploy.sh
#!/bin/bash
echo "Deploy to Production by $1"
```

```
#getResource.groovy
#!/usr/bin/env groovy
def call(String username){
    def deploy = libraryResource 'scripts/frontend/deploy.sh'
    writeFile(file: './deploy-script.sh', text: "${deploy}")
    sh "bash ./deploy-script.sh ${username}"
}
```

Shared Library

```
#deployContainer.groovy
#!/usr/bin/env groovy
import docker.release.Deployer
def call(Map config=[:]) {
    if (config.type == "slack") {
        echo Deployer.SLACK_MESSAGE
        echo config.message
    } else {
        echo Deployer.EMAIL_MESSAGE
        echo config.message
    }
}
```

```
#Deployer.groovy
#!/usr/bin/env groovy
package docker.release

class Deployer {
    static final String SLACK_MESSAGE = "Sending Slack Notification"
    static final String EMAIL_MESSAGE = "Sending Email"
}
```

Shared Library

```
library identifier: 'my-shared-library@main', retriever: modernSCM(  
  [$class: 'GitSCMSource',  
  remote: 'https://github.com/leangsengk90/my-shared-library.git',  
  credentialsId: ''])
```

```
//Configure in Configure System => Global Pipeline Library  
//@Library(['my-shared-library@main']) _
```

```
pipeline {  
  agent any  
  stages {  
    stage('Hello') {  
      steps {  
        helloWorld("DARA")  
        getResource('BONA')  
        script{  
          calculate.sum(12, 3)  
        }  
        deployContainer(type: 'slack')  
      }  
    }  
  }  
}
```



How to Clean Workspace

```
pipeline {
  agent any
  parameters{
    booleanParam(name: "IS_CLEAN", defaultValue: false, description: "Clean your workspace?")
  }
  stages {
    ... Mostly at the end ...
    stage('Clean Your Workspace') {
      when {
        expression{
          return "${param.IS_CLEAN}" == "true"
        }
      }
      steps {
        cleanWs()
      }
    }
    ...
  }
}
```



Pipeline with notify.events

- Install Plugin: Notify.Events

Plugin Manager

Updates Available **Installed** Advanced

Name ↓

[Notify.Events](#) 1.6.2

This plugin allows you to send messages from Jenkins jobs to Telegram, Viber, Skype, SMS, Voice Call and many others.
[Report an issue with this plugin](#)

Pipeline with notify.events

- Create Channels

Channels

Title

You have not create any channels yet.

Create

Create

Title

jenkins-notify

Group

☒ Allow public subscription [?]

Allow subscriptions to extra payable chats

Please pay attention that you must have a positive balance to use extra payable

☐ SMS

☐ Voice call

Allow subscription through integration

You have not configured any [integrations](#) yet

Save

Cancel

Pipeline with notify.events

- Add Sources and Choose CI/CD and Version control: **Jenkins**

The image shows two overlapping screenshots of the Jenkins web interface. The background screenshot is the 'jenkins-notify' configuration page, which has a header with a double-headed arrow icon and the text 'jenkins-notify'. Below this is a section titled 'Sources' with a lightbulb icon. Under 'Sources', there is a 'Title' field containing the text 'There are no channel sources yet' and a purple 'Add source' button. The foreground screenshot is the 'Add source' dialog, which has a lightbulb icon and the title 'Add source'. It features four category tabs: 'Monitoring' (bar chart icon), 'IoT and Smart Home' (house with signal icon), 'Website and E-Commerce' (shopping cart icon), and 'CI/CD and Version control' (link icon). The 'CI/CD and Version control' tab is selected. Below the tabs, there is a grid of source icons: AppCenter (red square with white icon), Bitbucket Cloud (blue square with white icon), CircleCI (black circle with white icon), CodeShip (blue circle with white icon), Drone (blue circle with white icon), Gitea (green circle with white icon), GitLab (orange square with white icon), Jenkins (Jenkins logo), TeamCity (black square with white 'TC' icon), and Travis-CI (yellow circle with white icon).

Pipeline with notify.events

- Add this script to pipeline:
 - `notifyEvents message: 'Hello
http://34.143.211.169:8080/job/${JOB_BASE_NAME}/${BUILD_NUMBER}/consoleText', token:
'gjRba5K3dLFan8VI7WkI3QK6n9eghQ-I'`

 **Add source**

Source

Jenkins v1

Title

Jenkins

Next

Cancel

 **Instruction**

Install the Notify.Events plugin according to the [instructions](#).

Jenkins build configuration:

Build step

- Add a build step
- Fill in the "Token": `gjRba5K3dLFan8VI7WkI3QK6n9eghQ-I`
- Fill in the "Message"

Post-build Actions

- Add a post build step
- Fill in the "Token" field: `gjRba5K3dLFan8VI7WkI3QK6n9eghQ-I`
- Fill in the "Message" (optional)
- Fill in the "Send when"

Done

Pipeline with notify.events

- Search for in Telegram: Notify.Events bot and Press “Start” and type: /subscribe _your-subscribe-code

The image shows a Jenkins Pipeline configuration interface. The 'Recipients' section is partially visible, showing a 'Title' field with the text 'There are no subscribers yet' and a 'Subscribe' button. The 'Channel subscription' section is the main focus, displaying a modal window for 'Telegram' notifications. The modal contains instructions for subscribing to notifications and for subscribing a group to notifications. It also lists 'Known limitations' and 'Available commands'. The background shows various notification channel options like Telegram, Signal, Slack, and Browser notifications.

Recipients

Title

There are no subscribers yet

Subscribe

Channel subscription

« jenkins-notify

Telegram

Subscribe to notifications

1. Add [Notify.Events bot](#) to your contact list.
2. Press "Start" to start getting notifications.

If you have already added the bot to your contact list, just run the `/subscribe _DrDwt-t` command inside the bot chat.

Subscribing a group to notifications

1. Add [Notify.Events bot](#) to your contact list.
2. Go to the bot profile and add it to your group.
3. Execute the `/subscribe _DrDwt-t @NotifyEventsBot` command inside the group chat.

Known limitations

Available commands

Telegram

Signal

Slack

Browser notifications

Discord

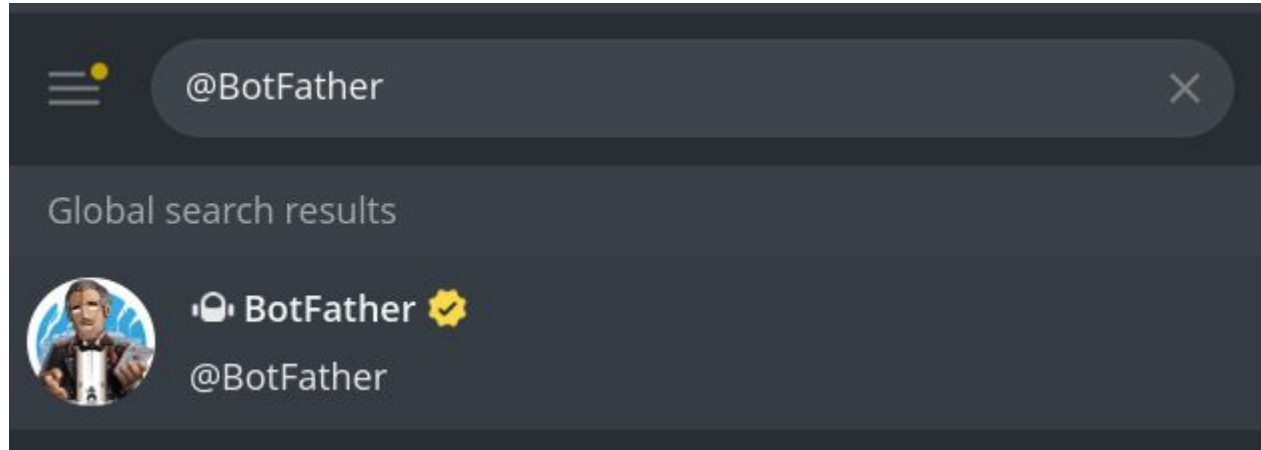
Voice call

Microsoft Teams Webhook

Twilio (SMS)

Slack Webhook

Pipeline with Telegram



Pipeline with Telegram

- Install Plugin: Telegram Bot
 - In Your Telegram:
 - Find @BotFather
 - Type: `/start => /newbot`
 - Enter your name: `dara`
 - Enter your username with “_bot”: `dara_bot`
 - Copy Token: `5871480698:AAHy9EcjCMN4jUCjEUqYCx-f-KoZNVU4Uzg`
 - Go to your new bot username: `dara_bot`
 - Type: `/start => /sub` (Type something in chat to get Chat ID)
 - Go to browser
 - URL: `https://api.telegram.org/bot5871480698:AAHy9EcjCMN4jUCjEUqYCx-f-KoZNVU4Uzg/getUpdates`
- ```
"chat": {
 "id": 456400937,
 "first_name": "Kao",
 "last_name": "Leangseng",
 "username": "kaoleangseng",
 "type": "private"
},
```

# Pipeline with Telegram

- Apply in Jenkins Pipeline
  - Document API: <https://core.telegram.org/bots/api#update>

```
post{
 always{
 sh 'curl -s -X POST
https://api.telegram.org/bot5871480698:AAHy9EcjCMN4jUCjEUqYCx-f-KoZNVU4Uzg/sendMessage -d
chat_id=456400937 -d text="Good Morning!"'
 }
}
```

A large, faint watermark of the ISTAD logo is centered in the background. It is a circular emblem with a blue outer ring containing the text 'INSTITUTE OF SCIENCE AND TECHNOLOGY ADVANCED DEVELOPMENT' and Khmer text. Inside the ring is a blue circle with a white atomic symbol and binary code '01010100 01000001 01000000'.

# Thank you

*Begin with the theory, Start with the practical*